

GATE CS/IT: Computer Networks

Flow Control

GATE Aspirant

July 23, 2026

What is Flow Control?

- **Definition:** A synchronization mechanism used to manage the rate of data transmission between two nodes to prevent a fast sender from overwhelming a slow receiver.
- **The Core Problem (Speed Mismatch):** If a sender transmits data faster than the receiver can process it (or faster than its buffer can store it), packets will be dropped, leading to retransmissions and network congestion.
- **Where it operates (OSI Model):**
 - **Data Link Layer:** Node-to-node flow control (e.g., between two adjacent routers).
 - **Transport Layer:** End-to-end flow control (e.g., TCP flow control between a client and a server).
- **Basic Mechanism:** The receiver provides feedback (Acknowledgments / Window Size) to the sender, dictating how much unacknowledged data the sender is allowed to transmit.

Approaches to Flow Control (GATE Focus)

- **Feedback-based Flow Control:** (Primarily tested in GATE)
 - The sender transmits frames and waits for an acknowledgment (ACK) from the receiver before sending more.
 - **Protocols:** Stop-and-Wait, Sliding Window (Go-Back-N, Selective Repeat).
- **Rate-based Flow Control:**
 - A built-in mechanism restricts the absolute rate (bits per second) at which the sender can transmit, without waiting for explicit ACKs for speed matching.
 - Often used in streaming/multimedia protocols.

Types of Delays in Computer Networks

- **Concept:** As a packet travels from a source to a destination, it suffers various delays at each node (router/switch) along the path.
- **Total Nodal Delay:** The sum of all individual delays at a single router.

$$D_{total} = D_{proc} + D_{queue} + D_{trans} + D_{prop}$$

- **Significance for GATE:** Understanding the exact difference between these delays—especially Transmission and Propagation—is the foundation for almost all Data Link Layer and TCP numericals.

1. Transmission & 2. Propagation Delays (GATE Core)

- **Transmission Delay (D_{trans}):** The time taken by the sender to put the entire packet onto the transmission medium.
 - Depends only on packet size and link bandwidth. Distance does *not* matter.
 - **Formula:** $D_{trans} = \frac{L}{B}$
 - L = Packet length (in bits), B = Bandwidth/Link capacity (in bps).
- **Propagation Delay (D_{prop}):** The time taken by one single bit to travel from the beginning of the link to the destination.
 - Depends only on distance and physical medium speed. Packet size does *not* matter.
 - **Formula:** $D_{prop} = \frac{d}{v}$
 - d = Distance (in meters), v = Velocity of signal (in m/s).

3. Processing & 4. Queuing Delays

- **Processing Delay (D_{proc}):**

- The time taken by a router to examine the packet header, determine the output link (routing), and check for bit-level errors.
- Usually in the order of microseconds. In GATE numericals, it is typically assumed to be **zero** unless explicitly given.

- **Queuing Delay (D_{queue}):**

- The time a packet spends waiting in the router's output buffer (queue) before it can be transmitted onto the link.
- Highly variable and depends on network congestion level.
- In most GATE deterministic numericals, this is also ignored unless the problem specifies a queuing model or buffer wait time.

The K, M, G Trap: Data Size vs. Bandwidth

- **Crucial GATE Pitfall:** The prefixes Kilo (K), Mega (M), and Giga (G) mean entirely different things depending on whether you are measuring **File Size (Data)** or **Link Speed (Bandwidth)**.

1. Data / Packet Size (Powers of 2)

- Used for memory, file sizes, buffer sizes, and packet lengths.
- Usually represented in **Bytes (B)**.
- **1 KB** (Kilobyte) = 2^{10} Bytes = 1,024 Bytes
- **1 MB** (Megabyte) = 2^{20} Bytes = 1,048,576 Bytes
- **1 GB** (Gigabyte) = 2^{30} Bytes

2. Bandwidth / Transmission Rate (Powers of 10)

- Used for link capacity, throughput, and clock rates.
- Usually represented in **bits per second (bps)**.
- **1 Kbps** (Kilobit/sec) = 10^3 bps = 1,000 bps
- **1 Mbps** (Megabit/sec) = 10^6 bps = 1,000,000 bps
- **1 Gbps** (Gigabit/sec) = 10^9 bps

Bits vs. Bytes: The Golden Rule for Numericals

- **The Notation Rule:**

- Uppercase '**B**' = Bytes.
- Lowercase '**b**' = bits.
- 1 Byte = 8 bits.

- **GATE Calculation Example:**

- **Given:** Packet size $L = 2$ KB, Bandwidth $B = 1$ Mbps.
- **Goal:** Calculate Transmission Delay ($D_{trans} = \frac{L}{B}$).
- **Step 1 (Convert everything to bits):**

$$L = 2 \times 2^{10} \text{ Bytes} = 2 \times 1024 \times 8 \text{ bits} = 16,384 \text{ bits}$$

- **Step 2 (Convert bandwidth using Base 10):**

$$B = 1 \times 10^6 \text{ bps} = 1,000,000 \text{ bps}$$

- **Step 3 (Calculate):**

$$D_{trans} = \frac{16,384}{1,000,000} \text{ seconds} = 16.384 \text{ ms}$$

- *Never* use 10^3 for KB, and *never* use 2^{20} for Mbps!

Introduction to Stop and Wait Protocol

- **Concept:** The simplest and most basic flow control protocol. It ensures the sender does not overwhelm the receiver by transmitting one frame at a time.
- **The Mechanism:**
 - ① The sender transmits exactly **one frame**.
 - ② The sender *stops* and waits for an acknowledgment (ACK).
 - ③ The receiver receives the frame, processes it, and sends an ACK back.
 - ④ Upon receiving the ACK, the sender transmits the *next* frame.
- **Window Sizes (Crucial for GATE):**
 - **Sender Window (W_s): 1** (can only send 1 frame without ACK).
 - **Receiver Window (W_r): 1** (expects exactly 1 specific frame).
- **Limitations:** Highly inefficient on links with high bandwidth and long distances (e.g., satellite links), because the sender spends most of its time waiting rather than transmitting.

Efficiency of Stop and Wait (GATE Core)

- To solve numericals, we must calculate the **Total Cycle Time** for one frame.
- Let T_t = Transmission Delay of Frame, and T_p = Propagation Delay.
- **Total Time (T_{cycle}):**

$$T_{cycle} = T_{t(\text{frame})} + T_{p(\text{frame})} + T_{t(\text{ACK})} + T_{p(\text{ACK})} + \text{Queuing/Processing}$$

- **Standard GATE Assumption:** Unless given in the question, assume $T_{t(\text{ACK})} \approx 0$ and Processing Delay ≈ 0 .
- Simplified T_{cycle} :

$$T_{cycle} = T_t + 2T_p$$

- **Efficiency (η):** The ratio of useful time (transmitting data) to total cycle time. Let $a = \frac{T_p}{T_t}$.

$$\eta = \frac{\text{Useful Time}}{\text{Total Cycle Time}} = \frac{T_t}{T_t + 2T_p} = \frac{1}{1 + 2a}$$

- **Throughput / Effective Bandwidth:** $\eta \times \text{Bandwidth}$

Stop and Wait: Condition for $> 50\%$ Efficiency

- **The Scenario:** In GATE numericals, you may be asked to find the conditions under which Stop and Wait performs reasonably well (i.e., efficiency $\eta > 50\%$).
- **Mathematical Derivation:**
- Start with the standard Efficiency formula:

$$\eta = \frac{T_t}{T_t + 2T_p}$$

- Set the condition for efficiency greater than 50% (0.5):

$$\frac{T_t}{T_t + 2T_p} > \frac{1}{2}$$

$$2 \times T_t > T_t + 2T_p$$

$$2T_t - T_t > 2T_p$$

$$T_t > 2T_p$$

- **Intuition:** For Stop and Wait to be at least 50% efficient, the time taken to push the frame onto the link (**Transmission Delay**) must be *strictly greater* than the Round Trip Time (**RTT = $2T_p$**) of the signal.

Minimum Frame Length for $> 50\%$ Efficiency

- **Recall the Condition:** We established that for efficiency $\eta > 50\%$, we require:

$$T_t > 2T_p$$

- **Expanding the Parameters:**

- **Transmission Delay (T_t):** $\frac{L}{B}$ (where L = Frame Length in bits, B = Bandwidth in bps)
- **Propagation Delay (T_p):** $\frac{d}{v}$ (where d = Distance in meters, v = Signal Velocity in m/s)

- **Substituting into the Inequality:**

$$\frac{L}{B} > 2 \times \left(\frac{d}{v} \right)$$

- **Condition for Frame Length (L):** Rearranging to solve for L :

$$L > \frac{2 \cdot d \cdot B}{v} \quad \text{or} \quad L > B \times \text{RTT}$$

- **GATE Insight:** The term $(B \times \text{RTT})$ is known as the **Bandwidth-Delay Product (BDP)**. It represents the maximum amount of data the link can hold in transit. For Stop and Wait to be $> 50\%$ efficient, the size of a single frame must physically exceed the pipe's capacity!

Stop and Wait Problem 1: Data Packet Loss

- **The Scenario:** The sender successfully transmits a data frame, but due to noise, congestion, or a faulty link, the frame is lost or dropped by a router before reaching the receiver.
- **The Deadlock (Infinite Waiting):**
 - The receiver does not know a frame was sent, so it continues waiting for a new frame.
 - The sender, following the Stop and Wait rule, has halted transmission and is waiting indefinitely for an ACK that will never come.
- **Impact:** The entire communication channel is paralyzed. The protocol fails to progress.

The Solution: Timeout Timer

- **The Mechanism:** To prevent infinite waiting, the sender equips itself with a **Timeout Timer**.
- **How it works:**
 - ① The sender starts the timer the moment it finishes transmitting a frame.
 - ② If an ACK arrives before the timer expires, the timer is stopped, and the sender proceeds to the next frame.
 - ③ If the timer expires (Timeout) and no ACK has arrived, the sender assumes the frame was lost and **retransmits** the same frame.
- **GATE Focus: Configuring the Timer (T_{out})**
 - The timer must wait long enough for a normal ACK to arrive.
 - **Formula:** $T_{out} \geq \text{Total Cycle Time}$

$$T_{out} \geq T_{t(\text{frame})} + 2T_p + T_{t(\text{ACK})} + T_{proc}$$

- In ideal GATE numericals (ignoring ACK transmission and processing time): $T_{out} > 2T_p$ (Strictly greater than the Round Trip Time).

Stop and Wait Problem 2: Lost Acknowledgment

- **The Scenario:** The sender transmits a frame, and the receiver successfully receives it. The receiver sends back an ACK, but **the ACK is lost** in transit.
- **The Chain of Events:**
 - ① The sender is waiting for the ACK. Its Timeout Timer expires.
 - ② Following the timeout rule, the sender assumes the data frame was lost and **retransmits** the exact same frame.
 - ③ The receiver gets the frame. Because the basic Stop and Wait protocol has no memory or ID for frames, the receiver assumes this is the *next, new* piece of data.
- **The Fatal Flaw (Data Duplication):** The receiver accepts the duplicate frame and passes it to the Network Layer. The data stream is now corrupted with repeated data!

The Solution: Frame Sequence Numbers

- **The Mechanism:** To solve duplication, the sender must attach a unique ID (Sequence Number) to the header of each data frame.
- **How it works:**
 - The receiver keeps track of the sequence number of the last successfully received frame.
 - When a frame arrives, the receiver checks its sequence number.
 - If the sequence number is *new*, accept the frame and send an ACK.
 - If the sequence number is a *duplicate* (already received), **discard the frame**, but **resend the ACK** (since the sender clearly didn't get the first ACK).
- **GATE Math: How many Sequence Numbers do we need?**
 - **Rule for any ARQ Protocol:** Minimum sequence numbers = $W_s + W_r$
 - In Stop and Wait, $W_s = 1$ and $W_r = 1$.
 - Minimum sequence numbers required = $1 + 1 = 2$.
 - We represent them using **Modulo-2 arithmetic (0 and 1)**.
 - **Bits required in header:** $\lceil \log_2(2) \rceil = 1$ bit.

Stop and Wait Problem 3: Delayed Acknowledgment

- **The Scenario:** The sender transmits Frame 0. The receiver gets it and sends an ACK. However, due to network congestion, the ACK is heavily **delayed** (but not lost).
- **The Chain of Events:**
 - ➊ The sender's Timeout Timer for Frame 0 expires before the ACK arrives.
 - ➋ The sender retransmits Frame 0.
 - ➌ The original delayed ACK finally arrives at the sender. The sender accepts it, assumes the retransmission was successful, and transmits the next frame (**Frame 1**).
 - ➍ Meanwhile, the receiver gets the retransmitted Frame 0, recognizes it as a duplicate (due to frame sequence numbers), discards it, and sends *another* ACK.
 - ➎ This *second* ACK arrives at the sender while it is waiting for an ACK for Frame 1.
- **The Fatal Flaw (Skipped Frames):** If ACKs do not have IDs, the sender assumes this second ACK is confirming Frame 1. The sender moves on to the next frame. **Frame 1 is completely lost** without the sender knowing!

The Solution: Acknowledgment Numbering

- **The Mechanism:** To solve the delayed ACK problem, the receiver must attach a Sequence Number to the Acknowledgments as well.
- **Standard ARQ Convention:**
 - An **ACK n** implies: "I have successfully received all frames up to $n - 1$, and I am now *expecting* Frame n ."
- **How it prevents the flaw:**
 - The receiver gets Frame 0 and sends **ACK 1** (expecting Frame 1). It gets delayed.
 - Sender times out, resends Frame 0.
 - The original delayed **ACK 1** arrives. Sender happily transmits Frame 1 and waits for **ACK 0**.
 - The receiver gets the duplicate Frame 0, discards it, and sends another **ACK 1** (still expecting Frame 1).
 - This second **ACK 1** arrives at the sender. Because the sender is currently waiting for **ACK 0**, it immediately recognizes the arriving **ACK 1** as a stale/delayed duplicate and safely **discards it**.
- **Result:** Frame 1 is protected, and the protocol is now completely robust!

GATE Numerical: Stop and Wait with Packet Loss

- **Problem Statement:** A sender uses the Stop and Wait protocol to transmit 10 data packets to a receiver. The channel has a predictable error rate where **every 4th packet transmitted is lost**. How many total transmissions are required to successfully send all 10 packets?
- **Mathematical Approach:**
 - Let the total number of physical transmissions be N .
 - Number of lost packets = $\lfloor \frac{N}{4} \rfloor$
 - Successful packets = Total Transmissions - Lost Packets
 - We need 10 successful packets: $N - \lfloor \frac{N}{4} \rfloor = 10$
- **Solving for N:**
 - Try $N = 12$: $12 - \lfloor \frac{12}{4} \rfloor = 12 - 3 = 9$ successful (Too short).
 - Try $N = 13$: $13 - \lfloor \frac{13}{4} \rfloor = 13 - 3 = 10$ successful.
- **Conclusion:** Exactly **13 transmissions** are required.

Visual Diagram: Transmission Trace

Tx Attempt	Packet ID	Channel Status	Total Successful
1	P1	Success	1
2	P2	Success	2
3	P3	Success	3
4	P4	LOST (4th Tx)	3
5	P4 (Retx)	Success	4
6	P5	Success	5
7	P6	Success	6
8	P7	LOST (8th Tx)	6
9	P7 (Retx)	Success	7
10	P8	Success	8
11	P9	Success	9
12	P10	LOST (12th Tx)	9
13	P10 (Retx)	Success	10 (Target Reached)

- *GATE Trick:* Notice that the packet lost is the one being transmitted at that exact sequence count, whether it is a new packet or a retransmission.

GATE Numerical: Probabilistic Packet Loss in S&W

- **Problem Statement:** A sender wants to successfully deliver **400 packets** to a receiver using the Stop and Wait protocol. The channel has an error probability $p = 0.2$ (meaning 20% of transmissions are lost). What is the expected total number of transmissions required?
- **Mathematical Intuition:**
 - Probability of packet loss (p) = 0.2
 - Probability of successful transmission ($1 - p$) = 0.8
- **Expected Transmissions per Packet:**
 - If a packet has a success rate of $(1 - p)$, the expected number of attempts to send that single packet successfully follows a geometric distribution.
 - The expected value (mean) for 1 successful delivery is:

$$E = \frac{1}{1 - p}$$

Calculation and GATE Shortcut

- **Total Transmissions Formula:** To send N packets successfully, we multiply the expected transmissions per packet by N :

$$\text{Total Transmissions} = N \times \left(\frac{1}{1-p} \right) = \frac{N}{1-p}$$

- **Applying the Values:**

$$\text{Total} = \frac{400}{1-0.2} = \frac{400}{0.8} = \frac{4000}{8} = \mathbf{500} \text{ transmissions}$$

- **Logical Verification Check:**

- Out of 500 total transmissions, exactly 20% will fail due to the error rate.
- Lost packets = $500 \times 0.2 = \mathbf{100}$ packets lost.
- Successful packets = $500 - 100 = \mathbf{400}$ packets. (*Matches our target perfectly!*)

Capacity of the Channel: Two Perspectives

- In Computer Networks (and specifically in GATE), "Channel Capacity" refers to two distinctly different concepts depending on the OSI layer you are analyzing.
- **1. The Data Link Layer Perspective (The "Pipe" Model):**
 - Refers to the physical volume of the link—how much data can be "in transit" at any given microsecond.
 - Used to calculate optimal window sizes for Flow Control protocols.
- **2. The Physical Layer Perspective (The Theoretical Limits):**
 - Refers to the absolute maximum data rate (in bps) that can be reliably transmitted over the medium based on analog bandwidth and noise.

1. Channel Capacity in Flow Control (The "Pipe" Model)

- **Concept:** Imagine the link as a physical water pipe. Bandwidth is the cross-sectional area (how much water enters per second), and Propagation Delay is the length of the pipe.
- **Link Capacity (One-Way):**

$$\text{Capacity (bits)} = \text{Bandwidth (bps)} \times \text{Propagation Delay } (T_p)$$

- **Round Trip Capacity / Bandwidth-Delay Product (BDP):**
 - In ARQ protocols (Stop & Wait, Sliding Window), the sender must wait for an ACK. Therefore, we care about how much data the sender can push onto the link during a full Round Trip Time (RTT).
 - **Formula:** $\text{BDP} = \text{Bandwidth} \times 2T_p$
 - *GATE Insight:* To achieve 100% link utilization in a Sliding Window protocol, the Sender Window Size (W_s) multiplied by the Frame Size (L) must be equal to or slightly greater than the BDP.
 $(W_s \times L \geq B \times 2T_p)$

2. Physical Layer Limits (Nyquist and Shannon)

- **Nyquist Theorem (For Noiseless Channels):**

- Defines the maximum bit rate purely based on the analog bandwidth (B_w in Hz) and the number of discrete signal levels (V).
- **Formula:** Bit Rate = $2 \times B_w \times \log_2(V)$ bps

- **Shannon Capacity Theorem (For Noisy Channels):**

- Defines the absolute theoretical upper limit of data rate for a channel subject to random white noise, regardless of how many signal levels you use.
- **Formula:** $C = B_w \times \log_2(1 + \text{SNR})$ bps
- *Note:* SNR is the Signal-to-Noise Ratio as a raw power fraction, not in Decibels (dB). If given in dB, convert it first: $\text{SNR}_{dB} = 10 \log_{10}(\text{SNR})$.

Motivation for Sliding Window: The Inefficiency of Stop & Wait

- **The Core Problem:** Stop and Wait is inherently a "ping-pong" protocol. The sender transmits exactly one frame and then goes completely idle while waiting for the ACK.
- **Wasted Channel Capacity:**
 - Recall the **Bandwidth-Delay Product (BDP)**. The link can hold $B \times 2T_p$ bits of data during one Round Trip Time.
 - In Stop and Wait, the sender only puts one frame (size L) into the link per RTT. If $L \ll \text{BDP}$, the pipe is mostly empty.
 - The sender is wasting valuable time and bandwidth just waiting.
- **The Worst-Case Scenario (Satellite Links):**
 - On links with high bandwidth and long physical distances, Transmission time (T_t) is very small, and Propagation time (T_p) is very large.
 - Efficiency $\eta = \frac{T_t}{T_t + 2T_p}$ plummets to near 0%.

The Solution: Pipelining (Sliding Window)

- **The Concept:** To fix the inefficiency, we must keep the sender busy. We do this by allowing the sender to transmit **multiple frames** back-to-back before requiring an ACK. This is called **Pipelining**.
- **The "Window":**
 - We define a **Sender Window Size** (W_s or N), which is the maximum number of unacknowledged frames the sender can transmit at once.
 - As ACKs arrive for older frames, the window "slides" forward, allowing the sender to push new frames into the pipe.
- **GATE Math: Achieving 100% Efficiency**
 - To keep the pipe completely full, the time taken to transmit N frames must be greater than or equal to the total Cycle Time.
 - $N \times T_t \geq T_t + 2T_p$
 - Dividing by T_t , we get the golden rule for 100% utilization:

$$N \geq 1 + 2a \quad (\text{where } a = \frac{T_p}{T_t})$$

Why Reuse Sequence Numbers?

- **The Constraint:** Every frame carries its sequence number in its header. Headers have a fixed, finite number of bits. We cannot count up to infinity; we must wrap around and reuse numbers.
- **Modulo Arithmetic:** If a protocol uses n bits for sequence numbering, the available sequence numbers range from 0 to $2^n - 1$.
 - Example: If $n = 3$ bits, sequence numbers are 0, 1, 2, 3, 4, 5, 6, 7, and then wrap back to 0.
- **The ARQ Golden Rule:** To ensure the receiver never confuses a delayed duplicate of an *old* frame with a *new* frame bearing the same reused sequence number, the total available sequence space must satisfy:

$$\text{Available Sequence Numbers} \geq W_s + W_r$$

- Let's see how this dictates the required bits (n) for the two main Sliding Window protocols.

The Need for Sequence Number Reuse

- **The Constraint:** Every frame carries its sequence number in its header. Because headers have a fixed, finite number of bits allocated, we cannot count upwards infinitely.
- **Modulo Arithmetic:** Sequence numbers must wrap around and be reused.
- If a protocol allocates n **bits** for the sequence number field:
 - The total number of unique sequence numbers available is 2^n .
 - The sequence numbers will range from **0** to $2^n - 1$.
 - Example: If $n = 3$ bits, the sequence space is $\{0, 1, 2, 3, 4, 5, 6, 7\}$. After frame 7, the next frame is numbered 0.

General Formula for Sequence Bits

- **The Danger of Wrapping Around:** The primary danger in sliding window protocols is that a severely delayed duplicate frame from an *old* window arrives just as the receiver is expecting a *new* frame that happens to reuse that exact same sequence number.
- **The Golden ARQ Rule:** To absolutely guarantee that the receiver never confuses an old duplicate with a new frame, the total available sequence space must be greater than or equal to the sum of the sender and receiver window sizes.

Available Sequence Numbers $\geq$ Sender Window (W_s) + Receiver Window (W_r)

$$2^n \geq W_s + W_r$$

- **Calculating the Number of Bits (n):** To find the minimum number of bits required in the header, we take the base-2 logarithm and round up (ceiling function):

$$n = \lceil \log_2(W_s + W_r) \rceil$$

Stop & Wait: A Theoretical Baseline

- **The Reality of Stop & Wait:** While S&W perfectly handles packet loss, lost ACKs, and delayed ACKs, it is fundamentally a *theoretical* protocol in modern networks.
- **Why it is impractical:** On high-bandwidth, long-distance links (like optical fiber or satellite), sending one frame and waiting for an ACK wastes almost 99% of the channel capacity.
- **The Practical Evolution:** To achieve real-world efficiency, networks utilize **Sliding Window Protocols** (pipelining).
- However, "Sliding Window" is just a general concept. To implement it practically, we must address how the receiver handles errors and out-of-order packets. This gives rise to two actual protocols:
 - ① **Go-Back-N (GBN)**
 - ② **Selective Repeat (SR)**

Design Considerations: GBN vs. Selective Repeat

- When designing a practical pipelined protocol, engineers must make trade-offs across three major components:
- **1. Receiver Window Size (W_r)**
 - *GBN*: $W_r = 1$. The receiver is simple and only accepts frames strictly in-order. Out-of-order frames are discarded.
 - *SR*: $W_r > 1$ (typically $W_r = W_s$). The receiver has memory/buffers to accept and store out-of-order frames.
- **2. Acknowledgment Strategy**
 - *GBN*: Uses **Cumulative ACKs**. An ACK n means "I have received all frames up to $n - 1$, send me n ."
 - *SR*: Uses **Independent ACKs**. An ACK specifically confirms only one individual frame.
- **3. Sequence Number Requirements**
 - *GBN*: Sequence space must be $2^n \geq W_s + 1$.
 - *SR*: Sequence space must be $2^n \geq W_s + W_r \implies 2^n \geq 2W_s$.

Introduction to Go-Back-N (GBN)

- **Concept:** A practical sliding window protocol that improves efficiency by pipelining, but keeps the receiver design extremely simple.
- **Window Sizes:**
 - **Sender Window (W_s):** N (Can transmit up to N unacknowledged frames).
 - **Receiver Window (W_r):** 1 (Can only accept frames strictly in sequential order).
- **Acknowledgment Strategy: Cumulative ACKs**
 - The receiver does not individually acknowledge every single frame.
 - An **ACK** k acts as a blanket statement: "I have successfully received all frames up to $k - 1$ in perfect order, and I am now waiting for frame k ."
 - If multiple frames arrive correctly in quick succession, the receiver can send a single cumulative ACK to acknowledge all of them at once.

The "Go-Back" Mechanism (Error Recovery)

- **The Scenario:** The sender transmits Frames 0, 1, 2, 3, and 4. Frame 1 is lost in transit, but Frames 2, 3, and 4 successfully reach the receiver.
- **Receiver Behavior (Ruthless Discarding):**
 - Because $W_r = 1$, the receiver is *only* expecting Frame 1.
 - When Frames 2, 3, and 4 arrive, they are out-of-order. The receiver has no buffer for them, so it **discards** them immediately.
- **Sender Behavior (The Penalty):**
 - The sender's timer for Frame 1 eventually expires (Timeout).
 - The sender realizes Frame 1 failed. Because it doesn't know the fate of the subsequent frames, it **"Goes Back"** to the unacknowledged frame.
 - It retransmits Frame 1, and then **retransmits all subsequent frames** (2, 3, and 4) that were previously in the window.
- **Trade-off:** Highly wasteful of channel bandwidth if error rates are high, but requires almost zero memory at the receiver side.

GATE Numerical: GB3 with Deterministic Loss

- **Problem Statement:** A sender wants to send **10 packets** using Go-Back-N with a window size of $N = 3$ (GB3). The channel drops **every 5th packet transmitted** (i.e., the 5th, 10th, 15th physical transmissions). How many total transmissions are required?
- **Understanding the GB3 Mechanism:**
 - The sender can transmit up to 3 packets back-to-back.
 - If packet k is lost, the receiver rejects all subsequent out-of-order packets.
 - The sender times out and must **retransmit packet k and everything sent after it** in that window.
 - In a continuous flow for $N = 3$, a single packet loss typically costs exactly **3 transmissions** (the lost packet + 2 subsequent packets that get discarded by the receiver).
- *GATE Trap Alert:* We only have 10 packets in total! When we reach the end of the data, the window won't have 3 packets left to send.

Visual Diagram: Transmission Trace (Part 1)

Tx Attempt	Packet ID	Channel Status	Receiver Action
1	P1	Success	Accepts P1 (Expects P2)
2	P2	Success	Accepts P2 (Expects P3)
3	P3	Success	Accepts P3 (Expects P4)
4	P4	Success	Accepts P4 (Expects P5)
5	P5	LOST (5th Tx)	-
6	P6	Success (but OOO)	Discards (Still expects P5)
7	P7	Success (but OOO)	Discards (Still expects P5)
— Timeout on P5. Sender goes back to P5 —			
8	P5 (Retx)	Success	Accepts P5 (Expects P6)
9	P6 (Retx)	Success	Accepts P6 (Expects P7)
10	P7 (Retx)	LOST (10th Tx)	-
11	P8	Success (but OOO)	Discards (Still expects P7)
12	P9	Success (but OOO)	Discards (Still expects P7)

Visual Diagram: Transmission Trace (Part 2)

Tx Attempt	Packet ID	Channel Status	Receiver Action
— Timeout on P7. Sender goes back to P7 —			
13	P7 (Retx)	Success	Accepts P7 (Expects P8)
14	P8 (Retx)	Success	Accepts P8 (Expects P9)
15	P9 (Retx)	LOST (15th Tx)	-
16	P10	Success (but OOO)	Discards (Still expects P9)
Notice: There is no P11. The window only contained P9 and P10. So only 2 packets were transmitted before the sender realizes P9 timed out!			
— Timeout on P9. Sender goes back to P9 —			
17	P9 (Retx)	Success	Accepts P9 (Expects P10)
18	P10 (Retx)	Success	Accepts P10 (Done!)

- **Final Answer:** It takes exactly **18 transmissions** to successfully send all 10 packets using Go-Back-3 under these error conditions.

- **Problem Statement:** A sender wants to send **10 packets** using Go-Back-N with a window size of $N = 4$ (GB4). The channel drops **every 6th packet transmitted** (i.e., the 6th, 12th, 18th attempts). How many total transmissions are required?
- **Understanding the GB4 Mechanism:**
 - The sender window size is 4. It can transmit up to 4 unacknowledged packets.
 - When a packet is lost, the receiver discards all subsequent packets.
 - The sender will continue transmitting until its window of 4 is full (or it runs out of packets to send), then it times out.
 - Upon timeout, it "goes back" and retransmits the lost packet and everything sent after it.
- Let's trace the transmissions step-by-step to see exactly how the window and timeouts behave.

Visual Diagram: Transmission Trace (Part 1)

Tx Attempt	Packet ID	Channel Status	Receiver Action
1	P1	Success	Accepts P1 (Expects P2)
2	P2	Success	Accepts P2 (Expects P3)
3	P3	Success	Accepts P3 (Expects P4)
4	P4	Success	Accepts P4 (Expects P5)
5	P5	Success	Accepts P5 (Expects P6)
6	P6	LOST (6th Tx)	-
7	P7	Success (but OOO)	Discards (Still expects P6)
8	P8	Success (but OOO)	Discards (Still expects P6)
9	P9	Success (but OOO)	Discards (Still expects P6)
<i>Window is full [P6, P7, P8, P9]. Timeout on P6. Sender goes back to P6.</i>			
10	P6 (Retx)	Success	Accepts P6 (Expects P7)
11	P7 (Retx)	Success	Accepts P7 (Expects P8)

Visual Diagram: Transmission Trace (Part 2)

Tx Attempt	Packet ID	Channel Status	Receiver Action
12	P8 (Retx)	LOST (12th Tx)	-
13	P9 (Retx)	Success (but OOO)	Discards (Still expects P8)
14	P10	Success (but OOO)	Discards (Still expects P8)
<i>Notice: Window allows [P8, P9, P10, P11]. However, P10 is the last packet. The sender stops at P10 and waits. Timeout on P8.</i>			
<i>— Timeout on P8. Sender goes back to P8 —</i>			
15	P8 (Retx)	Success	Accepts P8 (Expects P9)
16	P9 (Retx)	Success	Accepts P9 (Expects P10)
17	P10 (Retx)	Success	Accepts P10 (Done!)

- **Final Answer:** It takes exactly **17 transmissions** to successfully send all 10 packets using Go-Back-4 under these error conditions. (We never reach an 18th transmission, so a 3rd loss doesn't occur).

The Duplicate Packet Problem in GBN

- **The Setup:** Sequence numbers are represented using n bits, giving 2^n unique sequence numbers. What happens if we make the Sender Window Size exactly equal to the total sequence space ($W_s = 2^n$)?
- **Example Scenario:** Let $n = 2$ bits. Sequence numbers are $\{0, 1, 2, 3\}$. We set $W_s = 4$.
- **The Fatal Trace:**
 - 1 The sender transmits frames 0, 1, 2, and 3.
 - 2 The receiver successfully gets 0, 1, 2, and 3. The receiver's window slides forward. The next frame it expects wraps around to **Sequence 0**.
 - 3 The receiver sends an ACK, but **the ACK is lost** in the network.
 - 4 The sender times out. Following GBN rules, it retransmits its entire unacknowledged window: the *old* frames 0, 1, 2, and 3.
 - 5 **The Catastrophe:** The receiver is currently expecting **Sequence 0** (the new one). It receives the retransmitted **Sequence 0** (the old one). Because the numbers match perfectly, the receiver accepts the duplicate old data as fresh data!

The Solution: Bounding the Window Size

- **The Core Issue:** The receiver cannot distinguish between the first frame of the *current* window and the first frame of the *previous* window if their sequence numbers are identical.
- **The Fix (The GBN Golden Rule):** We must ensure the window size is strictly less than the total available sequence space.

$$W_s \leq 2^n - 1 \quad \text{or} \quad W_s < 2^n$$

- **Why this works (Revisiting the Example):**
 - Let $n = 2$ (Sequences $\{0, 1, 2, 3\}$). We restrict $W_s = 3$.
 - Sender transmits 0, 1, 2.
 - Receiver gets 0, 1, 2. The next sequence it expects is **Sequence 3**.
 - ACK is lost. Sender times out and retransmits 0, 1, 2.
 - The receiver gets Sequence 0. However, it is explicitly expecting **Sequence 3**.
 - Because $0 \neq 3$, the receiver instantly recognizes it as a delayed duplicate, **discards it safely**, and resends an ACK to rescue the sender. Data corruption is prevented!

Motivation for Selective Repeat: The Flaw in GBN

- **The GBN Penalty:** In Go-Back-N, the receiver is extremely simple ($W_r = 1$). If it expects Frame 2, and Frame 3 arrives, it ruthlessly drops Frame 3, even if it is perfectly intact.
- **The "Avalanche" of Retransmissions:**
 - If window size is 100, and packet #1 is lost, the sender will eventually have to retransmit packet #1 *and* packets #2 through #100.
 - A single bit error causes a massive waste of network bandwidth as 99 perfectly good packets are thrown away and resent.
- **When is GBN a bad choice?**
 - On links with **high error rates** (e.g., wireless links).
 - On links with a **large Bandwidth-Delay Product** (where the window size N is very large).

The Selective Repeat (SR) Solution

- **The Core Concept:** Instead of discarding out-of-order packets, the receiver should **buffer** them. The sender should then only retransmit the *specific* (selective) packet that was lost.
- **How SR changes the architecture:**
- **1. Receiver Window ($W_r > 1$):**
 - The receiver now has memory. Usually, $W_r = W_s$. It can accept and hold out-of-order frames until the missing frame arrives to fill the gap.
- **2. Independent Acknowledgments:**
 - Cumulative ACKs don't work well here. The receiver sends an ACK specifically for each individual frame it receives (e.g., "ACK 4" means "I got Frame 4", it says nothing about Frame 3).
- **The Trade-off:** SR significantly improves channel efficiency and throughput on noisy links, but at the cost of requiring more complex logic, sorting algorithms, and memory buffers at the receiver.

GATE Numerical: Efficiency in Selective Repeat

- **Problem Statement:** A network uses the Selective Repeat protocol for flow control. The sender window size is $W_s = 50$. The transmission delay for one frame is $T_t = 1$ ms, and the propagation delay is $T_p = 49.5$ ms. Calculate the channel efficiency.
- **Given Data:**
 - $W_s = 50$
 - $T_t = 1$ ms
 - $T_p = 49.5$ ms
- **Step 1: Find the parameter a**

$$a = \frac{T_p}{T_t} = \frac{49.5}{1} = 49.5$$

Step-by-Step Calculation

- **Step 2: Calculate Maximum Link Capacity (in frames)**

- To achieve 100% efficiency, the window size must be at least $1 + 2a$.

$$1 + 2a = 1 + 2(49.5) = 1 + 99 = \mathbf{100} \text{ frames}$$

- *Interpretation:* The channel can hold exactly 100 frames during one complete round trip.

- **Step 3: Calculate Efficiency (η)**

- Since our actual window size ($W_s = 50$) is less than the channel capacity (100), the efficiency is purely the ratio of what we transmit to what the pipe can hold.

$$\eta = \frac{W_s}{1 + 2a}$$
$$\eta = \frac{50}{100} = \mathbf{0.5} \text{ or } \mathbf{50\%}$$

- **GATE Insight:** Notice that the efficiency formula ($\frac{W}{1+2a}$) is identical for both Go-Back-N and Selective Repeat assuming an error-free channel. The protocols only perform differently when packets start getting lost!

The Receiver's Dilemma: Out-of-Order Data

- **The Core Problem:** The Network Layer (above the Data Link layer) demands packets in perfect, sequential order. Because Selective Repeat (SR) accepts packets out-of-order from the physical link, it must perform a **sorting and buffering task** before passing data upwards.
- **Anatomy of the Receiver Window (W_r):**
 - The window is an array of memory buffers of size W_r .
 - Each slot corresponds to a specific sequence number the receiver is legally allowed to accept.
 - The left edge of the window is the **Base** (the lowest sequence number not yet received).
- **When an Out-of-Order Packet Arrives:**
 - If packet k arrives and falls within the valid window ($Base < k \leq Base + W_r - 1$), the receiver accepts it.
 - It caches the packet in the specific buffer slot for k and marks it as "received".
 - It immediately sends an independent **ACK** k back to the sender.

The Sorting Task: Buffering and Sliding

- **The Blocking Condition:** Even if the receiver successfully buffers packets 2, 3, and 4, if packet 1 (the Base) is missing, the window **cannot slide**. The buffered packets are held hostage to maintain order.
- **Arrival of the Missing Packet (The Resolution):**
 - ① The delayed or retransmitted packet 1 finally arrives.
 - ② The receiver places it in the Base slot.
 - ③ **In-Order Delivery (Sorting):** The receiver scans its buffer from the Base upwards. It finds a contiguous block of received packets (e.g., 1, 2, 3, 4).
 - ④ It extracts this entire block, passes them to the Network Layer in perfect order, and empties those buffer slots.
- **Advancing the Window:**
 - The window rapidly "jumps" forward by the number of contiguous packets delivered.
 - The new Base becomes the sequence number of the next unreceived packet (in this example, packet 5).

Sender Window in Selective Repeat: The State Problem

- **The Core Difference:** In Go-Back-N, the sender window is simple: all packets are either unacknowledged or acknowledged consecutively. In Selective Repeat (SR), because ACKs are independent, the sender's window can develop "**holes**".
- **Packet States in the Window:** At any given time, slots in the sender window can be in one of three states:
 - 1 Sent, but *not yet* acknowledged (Timer running).
 - 2 Sent and *successfully* acknowledged (Hole/Gap).
 - 3 Not yet sent (but within the allowed window).
- **The Overhead:** Because packets can be acknowledged out of order, the sender cannot simply use one global timer. It must maintain a **logical timer for every single unacknowledged packet** in its window.

The Searching Task: Handling Incoming ACKs

- **When an ACK arrives (e.g., ACK 4):**
 - Unlike GBN (where ACK 4 clears everything up to 3), the SR sender must **search** its window buffer to find the exact slot corresponding to Sequence 4.
 - Once found, it marks that specific packet as "Acknowledged" and cancels its individual timer.
- **The Sliding Condition (The "Base" Check):**
 - Just like the receiver, the sender window has a **Base** (the oldest unacknowledged packet).
 - *If the ACK is for a packet in the middle of the window:* The sender marks it, but the window **cannot slide**.
 - *If the ACK is for the Base packet:* The sender marks it. Then, it **searches forward** through the window to find the next contiguous "unacknowledged" packet. The window rapidly slides forward, making this new packet the Base.

The Searching Task: Handling Timeouts

- **Timeout Trigger:** When a hardware interrupt signals that a timer has expired, the sender faces another searching task.
- **The Retransmission Search:**
 - The sender must search its timer records to identify *exactly* which sequence number timed out.
 - It then searches its transmit buffer for that specific packet.
 - It extracts **only that single packet**, pushes it back onto the transmission link, and restarts its specific timer.
- **GATE Takeaway:** Selective Repeat minimizes wasted bandwidth on the physical link, but it shifts the burden to the CPU and memory. Both the sender and receiver must constantly search, sort, and manage complex buffers!

GATE Numerical: Selective Repeat with Deterministic Loss

- **Context:** We will solve this using the **Selective Repeat (SR)** protocol to demonstrate the efficiency difference compared to Go-Back-N.
- **Problem Statement:** A sender wants to transmit **10 packets** using Selective Repeat with a window size of $W_s = 3$. The channel drops **every 5th packet transmitted** (i.e., 5th, 10th attempts).
- **Understanding the SR Mechanism:**
 - The sender transmits up to 3 packets. The window slides forward as soon as the *oldest* packet (Base) is acknowledged.
 - If a packet is lost, subsequent packets are still sent (if the window allows) and buffered by the receiver.
 - Upon timeout, the sender retransmits **only** the specific packet that timed out.

Visual Diagram: Transmission Trace (Part 1)

Tx Attempt	Packet ID	Channel Status	Receiver Window / Action
1	P1	Success	Delivers P1. (Expects P2)
2	P2	Success	Delivers P2. (Expects P3)
3	P3	Success	Delivers P3. (Expects P4)
4	P4	Success	Delivers P4. (Expects P5)
5	P5	LOST (5th Tx)	- (Base is still P5)
6	P6	Success (OOO)	Buffers P6. Sends ACK 6.
7	P7	Success (OOO)	Buffers P7. Sends ACK 7.
<i>Sender Window is now [P5, P6, P7]. P6 and P7 are marked ACKed, but Base (P5) is missing. The window cannot slide to P8. Sender times out on P5.</i>			
8	P5 (Retx)	Success	Gets P5. Delivers P5, P6, P7.

Visual Diagram: Transmission Trace (Part 2)

Tx Attempt	Packet ID	Channel Status	Receiver Window / Action
<i>Since P5, P6, and P7 are all acknowledged, the sender window rapidly slides forward to [P8, P9, P10].</i>			
9	P8	Success	Delivers P8. (Expects P9)
10	P9	LOST (10th Tx)	- (Base is still P9)
11	P10	Success (OOO)	Buffers P10. Sends ACK 10.
<i>Sender Window is [P9, P10, P11]. It has space for P11, but there are no more packets to send. Sender waits for P9 to timeout.</i>			
12	P9 (Retx)	Success	Gets P9. Delivers P9, P10 (Done!)

- **Final Answer:** It takes exactly **12 transmissions** to send 10 packets using Selective Repeat.
- **The Power of Selective Repeat:** Recall that under the *exact same conditions* (Window = 3, every 5th packet lost, 10 packets total), the Go-Back-N protocol required **18 transmissions**. Selective Repeat saved 6 unnecessary retransmissions by utilizing receiver buffers!

Sequence Numbers in Selective Repeat

- **The Universal ARQ Rule:** To prevent delayed duplicate packets from being mistaken as new packets, the total sequence space (2^n) must be greater than or equal to the sum of the sender and receiver window sizes.

$$2^n \geq W_s + W_r$$

- **Deriving the SR Equation:**

- In Selective Repeat, the receiver must buffer out-of-order frames, so the receiver window equals the sender window ($W_r = W_s$). Let's define this common maximum window size as N .
- Substituting into the universal rule:

$$2^n \geq N + N \implies 2^n \geq 2N$$

- **Maximum Window Size Limit:**

$$N \leq \frac{2^n}{2} \implies N \leq 2^{n-1}$$

- **Conclusion:** In Selective Repeat, the maximum window size can be at most **half** of the total available sequence space.

Why Must the Window be Half the Sequence Space?

- **The Violation Scenario:** Let $n = 2$ bits. Sequence numbers are $\{0, 1, 2, 3\}$. The rule dictates $W \leq 2$. What happens if we break the rule and set $W = 3$?
- **The Fatal Trace:**
 - ① **Tx:** Sender transmits packets 0, 1, and 2.
 - ② **Rx:** Receiver successfully gets 0, 1, and 2. Its window slides forward by 3 slots. It is now expecting sequences $[3, 0, 1]$.
 - ③ **Error:** All ACKs are lost in the network.
 - ④ **Timeout:** The sender times out and retransmits the *old* packet 0.
 - ⑤ **Corruption:** The receiver receives the old packet 0. Because its current active window $[3, 0, 1]$ is legally expecting a new sequence 0, it eagerly accepts the old duplicate as fresh data!
- **The Fix:** If we strictly follow $W = 2$, the receiver's window would slide from $[0, 1]$ to $[2, 3]$. The duplicate 0 would fall safely outside the window and be discarded.

ARQ Protocols Comparison (Part 1: Core Mechanics)

Feature	Stop & Wait	Go-Back-N (GBN)	Selective Repeat (SR)
Efficiency Formula	$\frac{1}{1+2a}$	$\frac{W}{1+2a}$	$\frac{W}{1+2a}$
Buffer Size	Sender: 1 Receiver: 1	Sender: N Receiver: 1	Sender: N Receiver: N
Sequence Numbers	$2^n \geq 1 + 1$ (Needs 1 bit: 0, 1)	$2^n \geq W_s + 1$	$2^n \geq 2W_s$
Acknowledgement	Independent (For single frame)	Cumulative (ACK n expects n)	Independent / Selective (ACK strictly for 1 frame)
Retransmission	Resends the single unacknowledged frame	Resends lost frame AND all subsequent frames	Resends ONLY the specific lost frame

ARQ Protocols Comparison (Part 2: Performance & Cost)

Feature	Stop & Wait	Go-Back-N (GBN)	Selective Repeat (SR)
Bandwidth	Highly wasted (Sender idle most of the time)	Efficient, but wastes bandwidth retransmitting good packets	Highly efficient (Zero redundant transmissions)
CPU / Overhead	Minimal (One timer, no sorting)	Low (Receiver simply drops out-of-order packets)	High (Requires individual timers, sorting algorithms)
Implementation	Extremely Simple	Moderate	Highly Complex